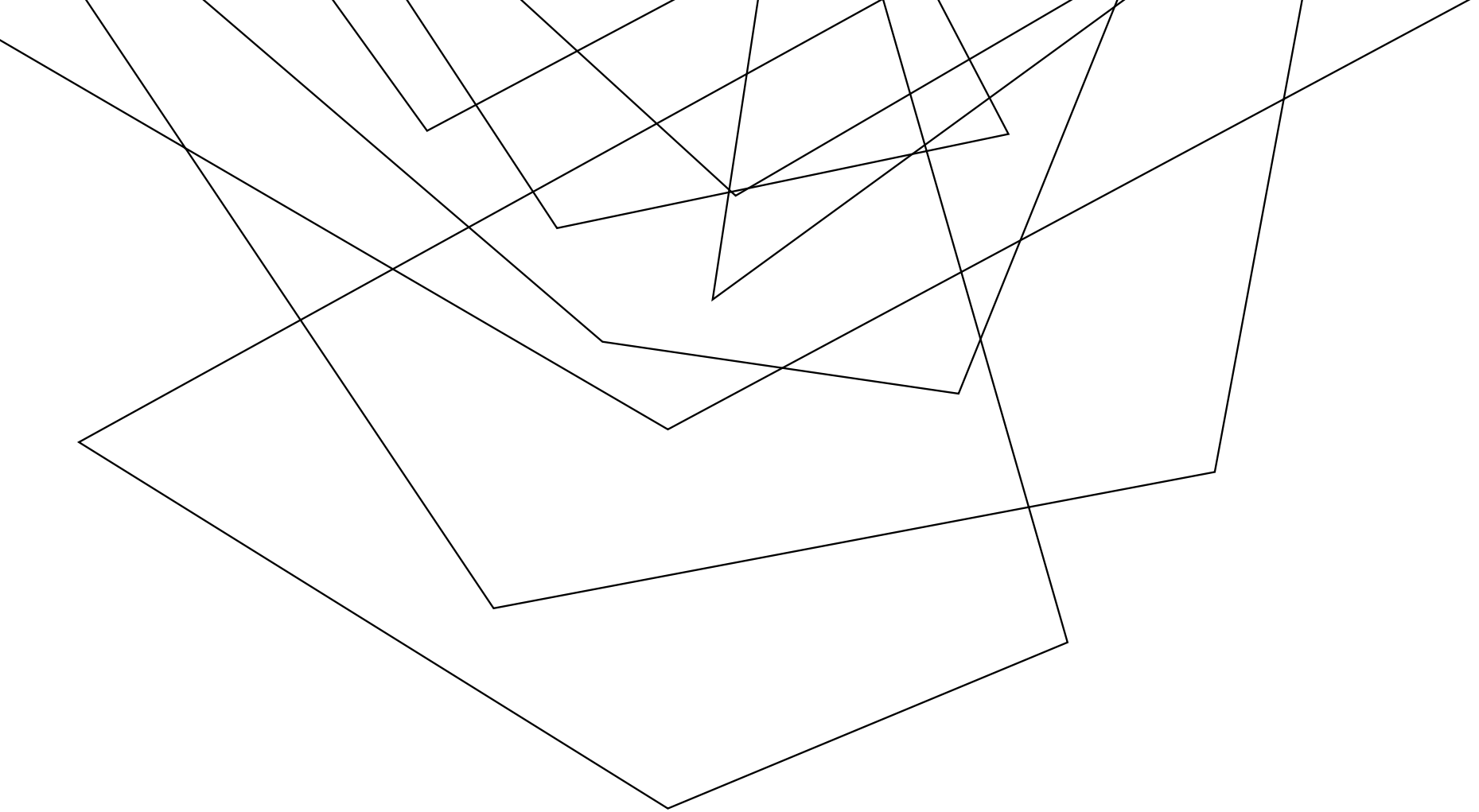

Graph-Mamba: Selective State Spaces for Long-Range Graph Modeling

- Achieving Global Context with Linear Efficiency

THE "EFFICIENCY VS. CONTEXT" TRADE-OFF

- **Graph Neural Networks (GNNs):** Great for local structure ($O(N)$), but suffer from *Over-smoothing* and fail at long-range tasks.

- **Graph Transformers:** Capture global dependencies perfectly, but have **Quadratic Complexity.**

- **The Bottleneck:** On large graphs (e.g., 1,000+ nodes), Transformers run out of memory (OOM).

- Current Proposed Solution is : Sparsification based on random walks

THE ENGINE: SELECTIVE STATE SPACE MODELS

Let's Understand The Terms

State – Compressed internal memory of any system.

Space – The Multi-dimensional vector where the State Lives (Eg. N-dimensional vector for N- features)

State Space Model – System that describes how the state changes in ts space according to the change in input.

CHALLENGES TACKLED BY GRAPH MAMBA

1. Non – sequential Graphs to Sequential SSM.

   - Heuristic Used – Node prioritization.

2. Tackle biasness due to node degree using permutation invariance.

3. Selective Nature (How much a particular node info to keep when the next input comes in).

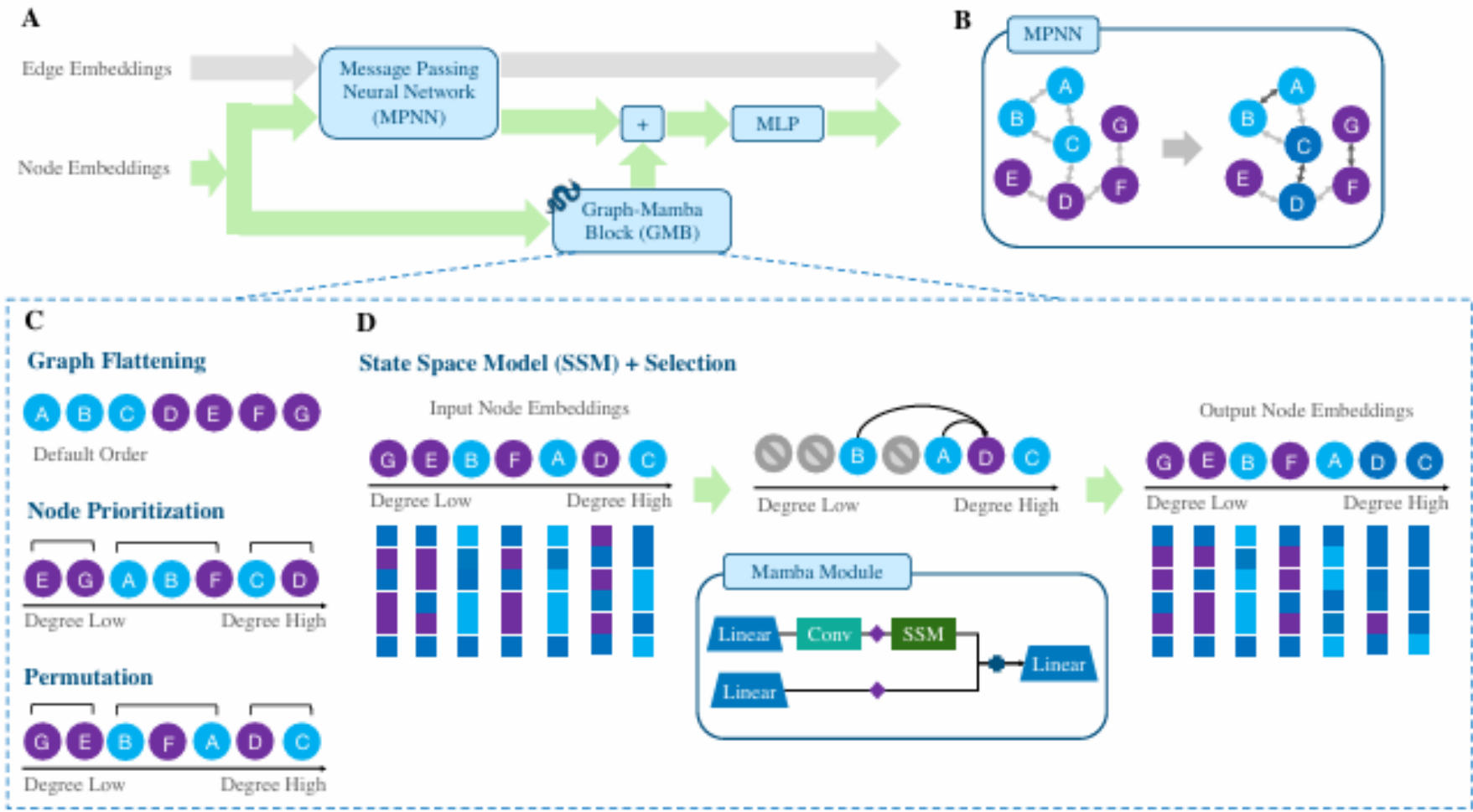

Figure 1. **Overview of Graph-Mamba architecture**, by incorporating GMB to replace the attention module in the GraphGPS framework. A) The GMB layer, an adaptation of GPS layer that combines an edge-based MPNN and a node-focused GMB to output updated node and edge embeddings. B) Graph-Mamba employs the GatedGCN model as default for MPNN. C) GMB's specialized training recipe with node prioritization and permutation techniques that perform informed graph sparsification. D) The selection mechanism with Mamba module that facilitates input-dependent context filtering.

RECURRENT SCANS & PERMUTATION

- SOLVING THE NON-SEQUENTIAL NATURE OF GRAPHS

•**The Problem:** Mamba needs a sequence. How do we order nodes?

•**Priority Strategy:** Sort nodes by **Degree (Node Prioritization)**.

•**Logic:** Important "hub" nodes are placed at the end of the sequence so they have the most "memory" of the entire graph.

- Problem – Biasness ??!!

•**Permutation Training:** Shuffling nodes with the same degree ensures the model doesn't overfit to one specific order.

- Multiple runs are done during training.
- Average the results and thus there is no biasness around any particular sequence.

HOW ARE RECURRENT SCANS DONE?

- **Recurrence:** Processes nodes sequentially via a Hidden State (h(t)).
- **The Formula:** h(t) = A h(t-1) + B h(x).
- **The "Selection" Innovation:** Parameters ($\Delta, B) are functions of the input x(t).

Term	Symbol	Dimensionality	Meaning in Graph-Mamba
Input Node	xt	(d)	The features of the current node in the scan.
Hidden State	ht	(d,N)	The total memory/context stored at step t.
State Matrix	A	(d,N)	How much memory to keep (system dynamics).
Input Matrix	B	(d,N)	How much of the new node to add to memory.

THE SELECTION : STEP SIZE

- HOW THE MODEL SAYS SELECTIVE

•Data – Dependent selection -> $g_t = \sigma(Linear(x_t))$

•How much info from previous state and how much from new input
-> greater for (high indegree)

$$h_t = (1 - g_t)h_{t-1} + g_t x_t,$$

- Achieved by Discretization of A and B

THE SELECTION : STEP SIZE

- HOW THE MODEL SAYS SELECTIVE

- Discretization of A and B –

- We cannot use the differential equation for state directly -> Graphs are discrete.

So, Transform A and B using **Zero – Order Hold**

Matrix A : The model takes the continuous "decay" rate and scales it by the step size (Delta). If (Delta) is very small, bar{A} is close to 1, meaning the model forgets almost nothing.

Matrix B: It determines how much "weight" the current node's features (x(t)) carry when they enter the hidden state.

$$\bar{A} = \exp(\Delta \cdot A)$$

$$\bar{B} = (\Delta \cdot A)^{-1} \left[\exp(\Delta \cdot A) - I \right] \cdot \Delta B$$

OVERALL UPDATE MECHANISM

Now, Actual update becomes -

$$h_t = \bar{A} h_{t-1} + \bar{B} x_t$$

Δ : Used for discretization is done for every node

↓

Decides which node to 'memorize', and to what extent.

$\bar{A}$ → calculated using A (transition matrix)

State dimension (N×N)

$\bar{B}$ → Input matrix

↓

Current node features

↓

Data-dependent.

COMPARION WITH EXISTING BENCHMARKS

Table 1. **Benchmark of Graph-Mamba on Long-Range Graph Datasets** with existing methods. These five datasets feature large input graphs with 150 to 1,400 nodes. Best results are colored in first, second, third.

Model	Peptides-Func AP ↑	Peptides-Struct MAE ↓	PascalVOC-SP F1 score ↑	COCO-SP F1 score ↑	MALNET-TINY Accuracy ↑
GCN	0.5930±0.0023	0.3496±0.0013	0.1268±0.0060	0.0841 ± 0.0010	0.8100
GIN	0.5498±0.0079	0.3547±0.0045	0.1265±0.0076	0.1339±0.0044	0.8898±0.0055
GatedGCN	0.5864±0.0077	0.3420±0.0013	0.2873±0.0219	0.2641±0.0045	0.9223±0.0065
GPS+Transformer	0.6575±0.0049	0.2510±0.0015	0.3689±0.0131	0.3774±0.0150	OOM (bs=8)
GPS+Performer	0.6475±0.0056	0.2558±0.0012	0.3724±0.0131	0.3761±0.0101	0.9264±0.0078
GPS+BigBird	0.5854±0.0079	0.2842±0.0130	0.2762±0.0069	0.2622±0.0008	0.9234±0.0034
Exphormer	0.6258±0.0092	0.2512±0.0025	0.3446±0.0064	0.3430±0.0108	0.9422±0.0024
Graph-Mamba	0.6739±0.0087	0.2478±0.0016	0.4191±0.0126	0.3960±0.0175	0.9340± 0.0027

THANK YOU